डॉ0 शकुन्तला मिश्रा राष्ट्रीय पुनर्वास विश्वविद्यालय, लखनऊ
Dr Shakuntala Misra National Rehabilitation University, Lucknow

Industrial Internship Report on

Banking Operations System

Prepared by

Lav Kush

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time. My project was on operations of Banks(How the banking operations done) This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

This report summarises my 4-week project based internship at UniConverge Technologies Pvt Ltd (UCT) in collaboration with upskill Campus and The IoT Academy. The internship was structured to bridge the gap between academic knowledge and industry requirements, giving me an opportunity to work on a real-world problem statement – **"Banking Operations System"**.

The project involved designing and implementing a fully functional banking application with a graphical user interface, supporting account creation, deposits, withdrawals, fund transfers, interest calculations, and transaction history export. The goal was to simulate core banking operations efficiently and securely.

I am grateful to UCT, upskill Campus, and The IoT Academy for providing this platform. Special thanks to team for their continuous guidance and support. This experience not only enhanced my technical skills but also improved my problem-solving and communication abilities.

To my juniors: I strongly encourage you to take up internships that challenge you to build complete projects – the learning is immense and will set you apart in your career.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

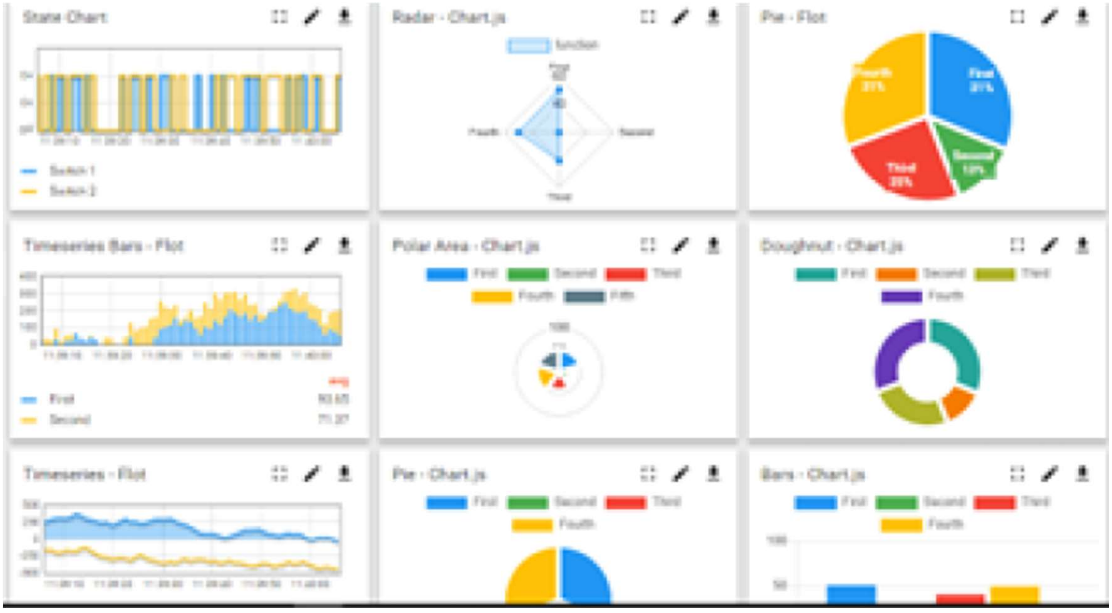
i. UCT IoT Platform (uct Insight)

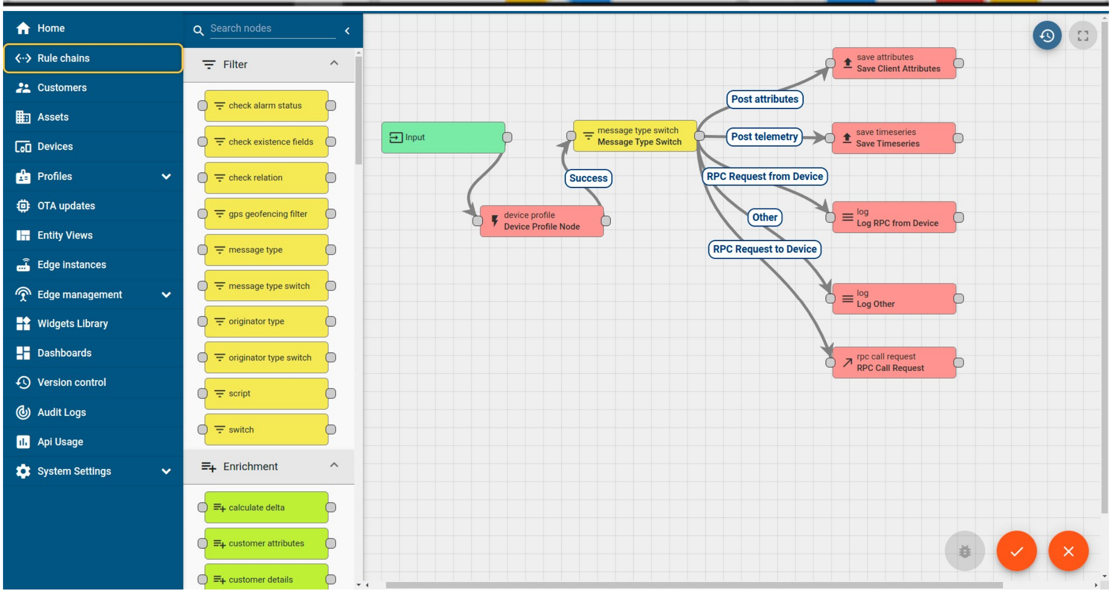
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





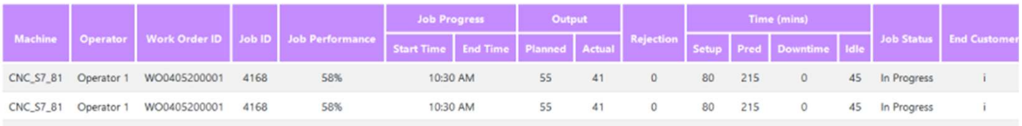
ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



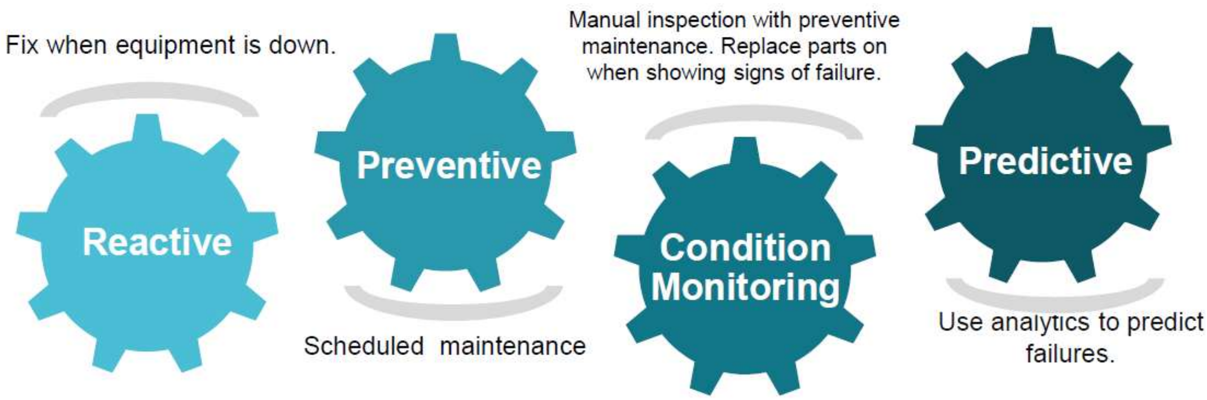
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

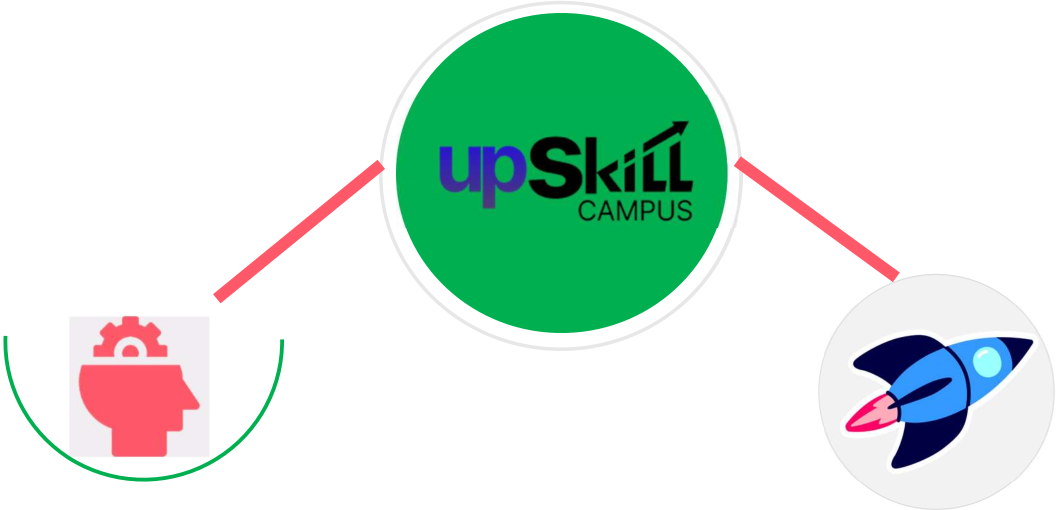
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

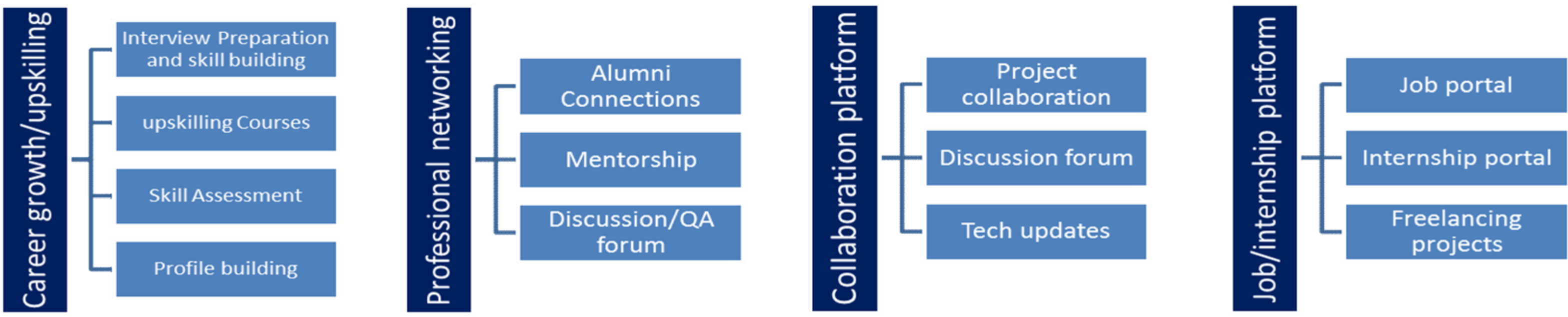
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

☛ To gain practical experience in designing a complete banking application.

☛ To implement key functionalities like account management, transaction processing, and interest calculation.

☛ To understand the importance of user authentication and data export.

☛ To learn version control (Git) and documentation best practices.

☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1]

[2]

[3]

2.6 Glossary

Terms	Acronym
GUI	
CSV	
OOP	

3 Problem Statement

Develop a Banking Information System that allows bank employees to manage customer accounts efficiently. The system should support:

- Creation of new accounts (Savings and Current) with initial deposits.
- Deposit and withdrawal operations with validation.
- Fund transfers between accounts.
- Viewing account details and transaction history.
- Application of interest on Savings accounts.
- Exporting all account data and transaction logs to CSV for reporting.
- Secure login access to prevent unauthorised use.

The solution should be user-friendly, reliable, and provide real-time feedback.

4 Existing and Proposed solution

Existing Solutions: Commercial banking systems like Finacle and TCS BaNCS are enterprise-grade solutions that support large-scale banking. However, they are expensive, require dedicated servers, and have steep learning curves. They are not suitable for small banking offices or educational purposes.

Proposed Solution: Our Banking Information System is a Java Swing-based desktop application that provides all core functionalities in a simple, intuitive GUI. It uses in-memory data storage (Map) for rapid prototyping and includes:

- Account creation with type selection (Savings/Current).
- Deposit/Withdraw with balance validation.
- Fund transfer with real-time updates.
- Interest application on Savings accounts (compound per annum).
- Export of all account details and transaction history to CSV.
- Login screen for basic security.

Value Addition: This system is lightweight, easy to deploy, and can be extended further. It serves as an excellent training tool and can be adapted for small-scale banking needs.

4.1 Code submission (Github link): [Check out]

4.2 Report submission (Github link) : https://github.com/Lav-developer/upskillcampus/blob/main/BankingInformationSystem_Lav_Kush_USC_UCT.pdf

5 Proposed Design/ Model

The system is built using Java Swing, which provides a rich set of UI components. The main interface is a `JTabbedPane` that organises functionalities into tabs. Each tab contains a dedicated panel with input fields and action buttons. Pop-up dialogs (`JOptionPane`) are used for error messages, confirmations, and success notifications.

The `BankAccount` class is the core data model, storing account number, holder name, type, balance, and a list of transaction strings. It also handles interest calculation based on type.

5.1 High Level Diagram (if applicable)

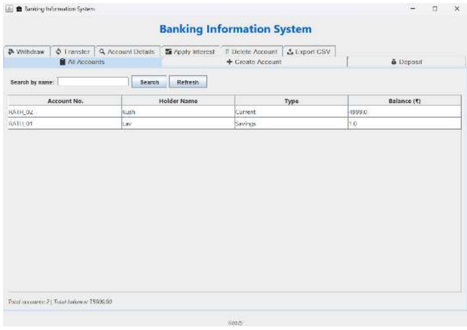
```
        +---------------------+
        |   Login Screen      |
        +----------+----------+
                   |
        +----------+----------+
        |  Main GUI (Tabs)    |
        +--+-------+-------+--+
           |       |       |
      +--v--+ +--v--+ +--v--+
      |Account| |Trans | |Admin|
      |Mgt   | |actions| |Tools|
      +------+ +-------+ +-----+
```

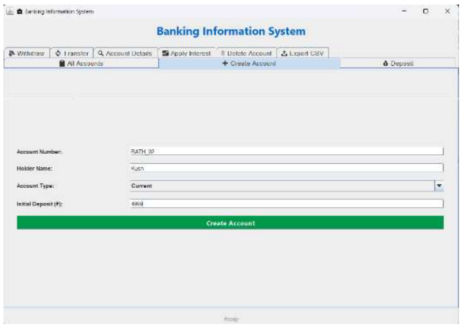
Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

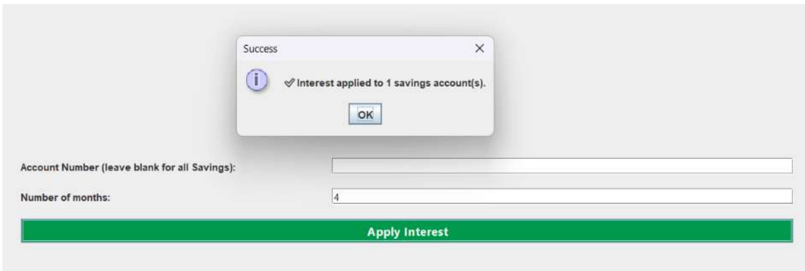
5.2 Low Level Diagram

- `BankingGUI` – main frame, contains all panels and event handlers.
- `BankAccount` – encapsulates account data, balance, transaction history, and methods for deposit/withdraw/interest.
- `DefaultTableModel` – used for the account list table.
- `TransactionAction` – functional interface for deposit/withdraw.

5.3 Interfaces







6 Performance Test

6.1 Test Plan/ Test Cases

- Test account creation with valid/invalid inputs.
- Test deposit/withdraw with positive/zero/negative amounts and insufficient balance.
- Test transfer between accounts with sufficient balance.
- Test interest application on Savings accounts for 1, 3, 6 months.
- Test CSV export with multiple accounts and transactions.
- Test login with correct/incorrect password.

6.2 Test Procedure

- Manual testing was performed by executing each operation and observing the UI feedback and table updates.
- Edge cases (e.g., empty fields, non-numeric amounts) were intentionally entered to verify error handling.

6.3 Performance Outcome

- The application responds instantly to user actions (sub-second latency) because all operations are in-memory.
- Memory usage is minimal (< 100 MB) as it only stores a few dozen accounts in a `HashMap`.
- CSV export completes within a second even for hundreds of transactions.
- For production scaling, we would replace the in-memory store with a database (e.g., MySQL) and implement connection pooling to handle concurrent users.
- The current design is suitable for a single-user environment (e.g., a bank teller's workstation). With a database backend, it can be extended to a multi-user system with transaction isolation.

7 My learnings

This internship was a transformative experience. On the technical front, I strengthened my Java programming skills, especially in GUI development using Swing, and learned to structure a multi-featured application with clean separation of concerns. Designing the BankAccount class and integrating it with the UI taught me the importance of modular code.

I also learned to handle user inputs gracefully, validate data, and provide meaningful feedback – essential for any production-grade application. Writing the report and maintaining a GitHub repository introduced me to version control and technical documentation, skills highly valued in the industry.

Beyond coding, I gained exposure to agile project planning, as I had to deliver the project in a fixed timeline. This experience boosted my confidence to take on complex challenges and communicate my solutions effectively. I am now better prepared for software development roles and internships in the future.

8 Future work scope

Though the current system fulfills the core requirements, there are several enhancements that can be added:

1. **Database Integration**: Replace the in-memory HashMap with a relational database (e.g., MySQL) to persist data across sessions and support multiple users concurrently.
2. **User Authentication**: Implement a proper user management system with roles (admin, teller) and salted password hashing.
3. **Transaction Logging**: Maintain a separate audit trail with timestamps and user IDs.
4. **Interest Schedules**: Allow users to set variable interest rates and auto-apply interest on a monthly/yearly basis.
5. **Reporting Dashboard**: Add graphical charts (e.g., pie chart of account types, balance trends) using JFreeChart or JavaFX.
6. **Web/Mobile Interface**: Convert the application to a web or mobile app using frameworks like Spring Boot and React Native.
7. **Advanced Security**: Add encryption for sensitive data (like account numbers and balances) and implement HTTPS for network communication.

These extensions would transform the application into a fully-fledged enterprise-grade banking system.